第 6-1 讲 • 从源码到安装包

> **模块 6 • 打包与发布** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成 1-3（构建三级流水线）、模块 5（质量与安全）

本节目标

读完并做完这一讲，你能：

1. 说清**"构建"和"打包"的区别**，以及为什么打包出来的东西有 116 MB 而代码只有 18.7 MB。
2. 看懂 `electron-builder.yml` 里**每一条排除规则**在省什么（含两个真实的踩坑记录）。
3. 理解 `asar` 是什么、**为什么原生模块必须解包。**
4. **自己出一个安装包**，并验证它真的能用。
5. 知道三种安装形式（`nsis` / `msi` / `portable`）的区别和适用场景。

一、构建 vs 打包

1.1 现实比喻：做菜 vs 装便当

回顾 1-3 讲的流水线，现在看最后一步：

阶段	比喻	产物	谁能用
写源码	备菜	`src\`	只有你
构建	炒菜装盘	`out\`	你自己吃（开发模式）
打包	**装便当盒 + 封膜 + 贴标签**	`release\`	**任何人**

"装便当"要多做四件事：

做的事	说明
带上一整套厨房	你的代码要在**别人的电脑**上跑，但那台电脑**没有 Node.js** → 所以要把 **Electron 运行时**（相当于一个精简版浏览器 + Node）一起打包
食材预处理	代码合并压缩进一个 `asar` 包
挑出能带和不能带的	`node_modules` 有 1 GB，其中大部分不用发（下一步详讲）
装成安装程序	生成能双击安装的 `.exe` / `.msi`

1.2 一个关键数字

我实测过两个数字，它们解释了 116 MB 是怎么来的：

东西	大小
你写的代码（打包成 `app.asar`）	**18.7 MB**
最终安装包	**116 MB**

也就是说：98 MB 不是你的代码，是"运行环境 + 资源"。

拆开看这 116 MB：

组成	大约	说明
Chromium + Node 运行时	~70 MB	Electron 本体（`icudtl.dat`、各种 `.dll`、`resources.pak`……）
你写的代码	18.7 MB	`app.asar`
OCR 模型	15.5 MB	4 个 `.onnx` 文件（5-3 讲讲过，为了本地识别）
原生模块（数据库引擎等）	~10 MB	`better-sqlite3`、`onnxruntime`

这就是为什么"一个记账软件怎么这么大" —— 因为它**自带了一整个运行环境。**

对比一下"网页版记账"：网页版不需要打包浏览器，因为用户已经有浏览器了。**代价是它必须联网、必须有服务器、数据不在你手里。**

这是一个典型的取舍：**体积换独立性和隐私。**

二、看懂打包配置

打开 `electron-builder.yml`。这个文件**每一行都有理由**，我逐段讲。

2.1 身份信息

```
appId: com.dailyledger.app
productName: 记账簿
copyright: 本地记账工具 · 数据全部保存在本机
```

字段	作用	能不能改
`appId`	**系统识别软件的身份证**（类似 `com.公司.产品`）	❌ **发布后不能改**（改了就变成新软件，用户数据对不上）
`productName`	**显示给用户的名字**（安装包文件名、程序名）	✅ 可以改
`copyright`	版权信息，显示在文件属性里	✅ 可以改

回顾 1-3 讲：你改 `productName` → 安装包文件名跟着变；但 `appId` 不变，所以**系统仍认为它是同一个软件**（用户数据不会丢）。

2.2 三条"不要重复干活"的配置

```
# 用本地已下载好的 Electron，避免再拉一次 100MB
electronDist: node_modules/electron/dist

# better-sqlite3 的 .node 已经按 Electron ABI 预编译好，禁止 electron-builder 重新编译
npmRebuild: false
```

这两条在解决同一个问题：别重复下载、别重复编译。

配置	防什么
`electronDist`	打包时**再下载一次 Electron**（100 MB，而且国内可能超时）。项目里已经有一份了，直接用
`npmRebuild: false`	打包时**重新编译原生模块**（`better-sqlite3` 需要编译，Windows 上经常失败）。项目里已经是编译好的，跳过

这是"踩过坑"才有的配置。新手常犯的错：以为打包就是跑一条命令，遇到编译失败就卡住。

2.3 排除规则（本讲重点）

```
files:
  - '**/*'                          # 默认：全都要
  - '!src/**'                       # 但不要源码（已经编译进 out 了）
  - '!docs/**'
  - '!scripts/**'
  - '!.tmp/**'
  - '!release/**'
```

逻辑：先"全都要"，再一条条划掉不需要的。

为什么不用"只列出需要的"：因为 `node_modules` 里的依赖关系太复杂，**漏一个就会运行时崩溃**。所以打包工具的默认策略是"全带 + 手工排除"。

两个真实的踩坑记录（配置里的注释就是证据）：

坑一：`分享包` 让安装包从 91 MB 涨到 1.26 GB

```
  # 分发件与压缩包不能进包：'**/*' 会把它们一起打进 asar
  # （踩过：分享包/ + 分享包.zip 让安装包从 91MB 涨到 1.26GB）
  - '!分享包/**'
  - '!*.zip'
```

发生了什么：项目的 `分享包\` 目录里放着要发给朋友的那些安装包，加上一个 589 MB 的旧 zip。`'**/*'` 把它们**一起打进了新的安装包——安装包里装着安装包。**

结果：91 MB 变成 **1.26 GB**。

教训：**打包前要问"这个目录会被一起打进去吗"**。特别是你自己放分发文件的地方。

坑二：`onnxruntime` 自带全平台二进制

```
# onnxruntime 自带全平台二进制，只留 Windows x64 的 CPU 版：
# darwin / linux / arm64 / DirectML 这些加起来 200MB+，这台机器和分发对象都用不上
- '!node_modules/onnxruntime-node/bin/napi-v6/darwin/**'
- '!node_modules/onnxruntime-node/bin/napi-v6/linux/**'
- '!node_modules/onnxruntime-node/bin/napi-v6/win32/arm64/**'
- '!node_modules/onnxruntime-node/bin/napi-v6/win32/x64/DirectML.dll'
```

问题：`onnxruntime-node` 这个库**为了方便，把 macOS / Linux / ARM / 各种 GPU 加速的二进制全放进来了。**

我实测了各平台的真实体积：

平台目录	体积
`win32`（Windows，**要用的**）	133.3 MB
`darwin`（macOS，**用不上**）	**85.3 MB**
`linux`（Linux，**用不上**）	**68.4 MB**
合计白带	**153.7 MB**

处理：**只留 Windows x64**，其余全排除。**省下 153.7 MB**（还要加上 DirectML 那几个 GPU 加速的 DLL）。

这条特别值得学：**第三方库的体积，往往比你的代码大得多。** 打包前**看一眼产物构成**，经常能省下几百 MB。

你可以自己验证这条规则的效果：

```
# 源目录里有几个平台，各多大
Get-ChildItem node_modules\onnxruntime-node\bin\napi-v6 -Directory | ForEach-Object {
  $size = (Get-ChildItem $_.FullName -Recurse -File -EA SilentlyContinue | Measure-Object
  "{0,-10} {1,8} MB" -f $_.Name, [math]::Round($size/1MB,1)
}
```

预期看到三个平台（`darwin`、`linux`、`win32`），其中两个是白带的。

2.4 随包发的资源

```
extraResources:
  - from: resources/icons
    to: icons
  # 账单截图识别用的 PP-OCRv4 模型（约 15MB）：随包发，装完即可离线使用
  - from: resources/ocr
    to: ocr
```

`extraResources` 和 `files` 的区别：

	`files`	`extraResources`
放进哪	`app.asar`（代码包里）	`resources\` 目录（**asar 外面**）
适合放什么	代码、依赖	**需要按路径直接访问的文件**（图标、模型）
为什么	打包压缩，加载快	有些库**只认真实文件路径**，读不了 asar 内部

OCR 模型为什么要放 asar 外面：因为 `onnxruntime` 需要**真实文件路径**去加载模型。**如果压进 asar，它找不到。**

2.5 原生模块必须解包（这条是硬性要求）

```
# 原生模块必须解包到 asar 外，否则 require 不进来
asarUnpack:
  - '**/node_modules/better-sqlite3/**'
  - '**/node_modules/onnxruntime-node/**'
```

先解释 `asar` 是什么：

`asar` 是 Electron 自己的"打包格式"——它把成千上万个文件合并成**一个文件**（`app.asar`），好处是：

- 加载快（不用一个个开文件）
- 文件数少（Windows 上复制几千个小文件很慢）

但 .node 文件必须解包：

`better-sqlite3` 和 `onnxruntime` 里包含 `.node` **文件**（编译好的原生二进制）。**操作系统的加载器只能加载真实文件，读不了 asar 内部的虚拟文件。**

所以：这几个包必须**解包到 `app.asar.unpacked\` 目录**（asar 外面）。

你可以自己确认：

```
Get-ChildItem release\win-unpacked\resources -Force | Select-Object Name
Get-ChildItem release\win-unpacked\resources\app.asar.unpacked\node_modules -Directory | S
```

预期看到：

```
resources\
├─ app.asar                    ← 你的代码（压成一个文件）
├─ app.asar.unpacked\          ← 解包出来的原生模块
│   └─ node_modules\
│       ├─ better-sqlite3\     ← 数据库引擎
│       └─ onnxruntime-node\   ← OCR 推理引擎
├─ icons\
└─ ocr\
```

这张图就是"打包产物长什么样"的完整答案。

三、三种安装形式

```
win:
  target:
    - target: nsis       # 安装程序
    - target: msi        # MSI 安装包
    - target: portable   # 免安装版
```

形式	产物	适合谁	特点
nsis	`daily-ledger-0.1.0-setup.exe`	**大多数用户**	有安装向导、能选目录、建快捷方式、能卸载
msi	`daily-ledger-0.1.0-installer.msi`	企业部署	系统级安装包，支持命令行静默安装、组策略推送
portable	`daily-ledger-0.1.0-portable.exe`	**想试试的人**	双击就跑，**不写注册表**，删文件即卸载

`nsis` 的几条配置值得看：

```
nsis:
  oneClick: false                          # 不用"一键安装"，走向导（让用户能选目录）
  perMachine: false                        # ★ 只给当前用户装，不需要管理员权限
  allowToChangeInstallationDirectory: true
  createDesktopShortcut: true
  createStartMenuShortcut: true
```

★ **`perMachine: false` 这一条很重要**：

设置	效果
`perMachine: true`	装到 `Program Files`，**需要管理员权限**（会弹 UAC）
`perMachine: false`	装到 `%LOCALAPPDATA%\Programs`，**不需要管理员权限**

为什么选 false：**降低安装门槛**。用户双击就能装，不用面对"是否允许此应用更改你的设备"那个吓人的弹窗。

代价：多用户登录同一台电脑时，每个人要各装一次。**对个人记账软件来说，这个代价可以接受。**

四、动手任务

步骤 1：看懂产物结构

```
cd "C:\Users\34123\Desktop\记账簿"

# ① 三个安装包
Get-ChildItem release -File | Where-Object { $_.Extension -in '.exe','.msi' } |
  Select-Object Name, @{n='MB';e={[math]::Round($_.Length/1MB,1)}}

# ② 解包后的真实程序目录
Get-ChildItem release\win-unpacked -Force | Select-Object Name

# ③ 你的代码有多大
"app.asar: " + [math]::Round((Get-Item release\win-unpacked\resources\app.asar).Length/1MB

# ④ 整个未打包目录有多大（对比用）
"win-unpacked 总计: " + [math]::Round((Get-ChildItem release\win-unpacked -Recurse -File |
```

预期：

项	数值
setup.exe	116.3 MB
msi	127.5 MB
portable.exe	116.1 MB
app.asar（你的代码）	**18.7 MB**
win-unpacked（解压后）	约 346 MB

记住这组数字的对比：**代码 18.7 MB，安装包 116 MB，解压后 346 MB。**

为什么安装包比解压后小：因为安装包是**压缩过的**（就像 zip）。用户装完占 346 MB——**这就是为什么软件"装完变大"。**

步骤 2：验证排除规则的效果

任务：确认那 200 MB 的其他平台二进制**没有**进安装包。

```
# ① 源目录里有哪些平台
Get-ChildItem node_modules\onnxruntime-node\bin\napi-v6 -Directory | Select-Object Name

# ② 打包产物里只剩哪些
Get-ChildItem release\win-unpacked\resources\app.asar.unpacked\node_modules\onnxruntime-no
```

预期：

- 源目录：`darwin`、`linux`、`win32`（可能还有别的）
- **打包产物：只有 `win32`**

这就是"排除规则生效"的证据。 而它省下了约 200 MB。

步骤 3：自己出一个安装包（本讲核心）

这一步会真的跑打包，需要几分钟。

① 先确认环境正常：

```
npm run typecheck
```

预期：0 错误。

② **打包**（这一步会构建 + 打包，约 2–4 分钟）：

```
npm run build:win
```

预期最后看到：

```
打包完成，产物在 release/
```

③ **检查产物**：

```
Get-ChildItem release -File | Where-Object { $_.Extension -in '.exe','.msi' } |
  Select-Object Name, @{n='MB';e={[math]::Round($_.Length/1MB,1)}}, LastWriteTime
```

预期：三个文件的**修改时间变成刚刚。**

④ **如果你看到下载工具链超时**：

`build-win.mjs` 里已经配了国内镜像（源码注释说明了：

> 这台机器直连 GitHub 下载 electron-builder 的工具链会超时，所以默认走 npmmirror 镜像

如果还是失败，手工指定：

```
$env:ELECTRON_BUILDER_BINARIES_MIRROR="https://npmmirror.com/mirrors/electron-builder-bina
npm run build:win
```

步骤 4：验证安装包真的能用（最重要的一步）

"打包成功"不等于"能装能用"。 项目里有一个专门的验证：

```
npm run verify:packaged 2>&1 | Select-String "通过|FAIL"
```

预期：**19/19 通过。**

它验证了什么（这是真实检查项）：

检查	说明
产物存在	三个文件都在
托盘行为	托盘创建、点 X 隐藏、从托盘唤回、进程不退出
能写入记录	用打包后的 exe 真的写一笔进沙箱数据库
重启后数据还在	关掉再开，数据没丢
免安装版能启动	并读到同一份数据
版本号正确	exe 属性里的 ProductVersion 与预期一致

这就是"打包产物验证"的价值：**它测的是最终交付给用户的那个东西**，不是源码。

为什么不能用"源码能跑"代替：因为打包会改变很多东西（代码被合并压缩、原生模块被解包、资源换了位置）。**这些变化都可能在打包后才暴露问题。**

步骤 5（可选）：真的装一次

如果你想体验完整流程：

1. 双击 `release\daily-ledger-0.1.0-setup.exe`
2. **会看到 Windows 蓝色警告**（因为没买代码签名证书）
3. 点「更多信息」→「仍要运行」
4. 跟着向导装完

注意：你**已经装过**这个软件（`%LOCALAPPDATA%\Programs\daily-ledger`），所以装的时候可能会覆盖它。**你的记账数据不受影响**（数据在 `D:\加速器`，和程序目录分开——1-2 讲学过）。

装完可以卸载（设置 → 应用 → 记账簿 → 卸载），或者留着。

卡住了怎么办

现象	原因	怎么办
打包超时/下载失败	工具链下载慢	用步骤 3 ④ 的镜像环境变量
报错 `Cannot find module electron-builder`	依赖没装	`npm install --ignore-scripts`
打包成功但 exe 打不开	可能缺原生模块	看 `app.asar.unpacked` 里有没有 `better-sqlite3`
安装包体积异常大（几百 MB）	有不该进包的东西	对比 `files` 排除规则，看是不是新增了大目录
不想真的装	完全理解	只跑 `verify:packaged` 就够了（它在沙箱里验证）

五、自测题

1. "构建"和"打包"的区别是什么？为什么安装包有 116 MB 而代码只有 18.7 MB？

2. `asar` 是什么？为什么 `better-sqlite3` 必须解包到 asar 外面？

3. `分享包\` 让安装包从 91 MB 涨到 1.26 GB，这件事的教训是什么？

4. `perMachine: false` 意味着什么？为什么这么选？

5. 为什么"源码能跑"不能代替 `verify:packaged`？

答案与解析

1. **构建**把源码变成可运行产物（`out\`），**打包**把产物 + 运行环境 + 资源装成安装程序（`release\`）。→ 因为打包要**带上一整套运行环境**：Chromium + Node（约 70 MB）、OCR 模型（15.5 MB）、原生模块（约 10 MB）。**你的代码只占 18.7 MB。**

2. `asar` 是 Electron 的打包格式，把成千上万个文件合并成一个文件（加载快、文件数少）。 → 但 `better-sqlite3` 里含 `.node` **原生二进制，操作系统的加载器只能加载真实文件**，读不了 asar 内部的虚拟文件。所以必须 `asarUnpack` 解包到 `app.asar.unpacked\`。

3. 打包前要检查"这个目录会不会被一起打进去"。 → `files: '**/*'` 会把项目里所有东西都带上，包括你自己放分发文件的地方。**结果是"安装包里装着安装包"，体积暴涨 14 倍。**

4. 意味着**装到用户目录（`%LOCALAPPDATA%`）而不是** `Program Files`，因此**不需要管理员权限**（不会弹 UAC）。 → 好处是**降低安装门槛**；代价是多用户共用一台电脑时要各装一次。对个人记账软件可以接受。

5. 因为**打包会改变很多东西**：代码被合并压缩、原生模块被解包到 asar 外、资源换了位置。 → 这些变化**只在打包后才暴露问题**。所以必须**测最终交付的那个东西**（用打包后的 exe 真跑一遍）。

六、术语表

术语	一句话解释
打包 / Package	把程序 + 运行环境 + 资源装成安装程序
asar	Electron 的归档格式，把很多文件合并成一个
`app.asar`	打包后的代码归档（实测 18.7 MB）
`asarUnpack`	把指定内容解包到 asar 外面（原生模块必须）
原生模块 / Native Module	用 C++ 编译的 `.node` 二进制，如数据库引擎
`extraResources`	额外资源，放进 asar 外的 `resources\` 目录
nsis	常见 Windows 安装程序格式
msi	微软系统级安装包格式
portable	免安装版，双击即用
`perMachine`	是否给整台机器安装（需要管理员权限）
UAC	Windows 的用户账户控制弹窗
排除规则	`files` 里 `!` 开头的条目，用来剔除不需要的文件

七、本讲与项目的对应

- 你查看的文件：`electron-builder.yml`、`scripts\build-win.mjs`、`release\` 产物结构
- 你执行的命令：`npm run build:win`（打包）、`npm run verify:packaged`（验证产物）
- **打包不影响你的记账数据**（数据在 `D:\加速器`，与程序目录分离）
- 上一讲：5-3 隐私与离线 ｜ 下一讲：**6-2 发布：网站、下载、被搜到**

讲义版本：v1 ｜ 对应项目版本：0.1.0